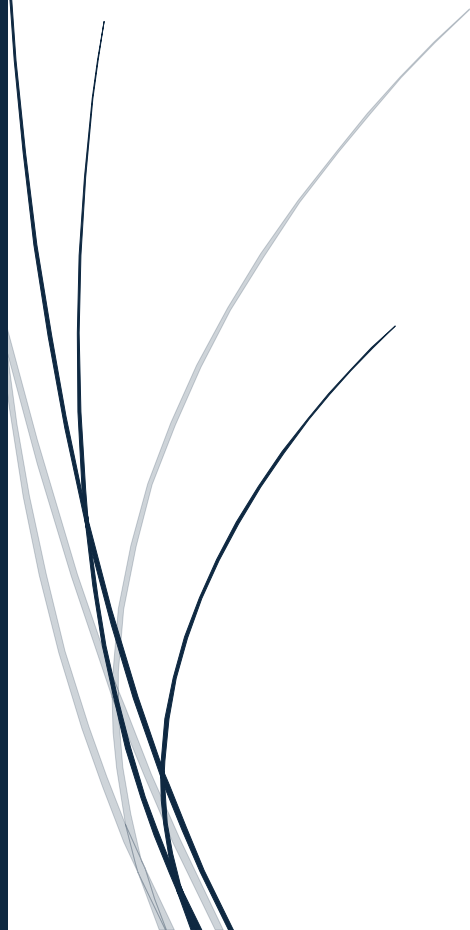


9/28/2024

Developer Guide

The System Architecture and
Components



TM MONARE
MB NDLOVU
N TSHABALALA
PV VUKEA

TEAM 4

Developer Guide: System Architecture and Components

1. Overview

The AutoWorx Barnyard Workshop Management System is designed to streamline and optimize the management of automotive services by providing tools for handling appointments, invoices, service tracking, and customer management. The system comprises two main components: a Mobile App for customers and a Web App for mechanics and workshop administrators. This guide provides an overview of the system's architecture, the key components, and how they interact to deliver the required functionality.

2. System Architecture

The architecture of the system follows a client-server model using a RESTful API for communication between the frontend (mobile and web apps) and the backend server. The system is structured as follows:

- Frontend (Presentation Layer)

- Customer Mobile App: Developed using Flutter, this app allows customers to book appointments, track service progress, view invoices, and access their service history.
- Mechanic Mobile App: This is a feature that allows the mechanic to log into the app, view their assigned Tasks and update tasks reports.
- Manager Web App: Built with React, the web app provides tools for mechanics and workshop administrators to manage appointments, manage tasks and services, and handle invoices.

- Backend (Business Logic Layer)

- Django REST Framework (DRF): The backend API is implemented using Django, exposing endpoints to manage customers, vehicles, services, and invoices. It handles all core business logic, ensuring secure data flow and validation between the frontend and the database.

- Database (Data Layer)

- PostgreSQL: The relational database stores all critical data, including customer information, vehicles, service records, appointments, invoices, and users.

- Authentication & Security

- Role-Based Access Control: Different user roles (customers, mechanics, administrators) determine what data and actions each user can access and perform.

3. Key Components

The system consists of several modular components that interact through the API:

a. User Management

- Registration & Login: Allows users (customers and mechanics) to create accounts and log in using secure authentication methods. Passwords are hashed and stored securely.
- Profile Management: Customers can manage their personal and vehicle information, while mechanics can manage service-related tasks. Managers Can Add new mechanics via the web app

b. Appointment Management

- Customers can schedule service appointments using the mobile app, selecting a service type, date, and time.
- Manager can view upcoming appointments and manage the status of each service.

c. Service Management

- Mechanics track the status of vehicle services through various stages (Inspection, In Progress, Completed).
- The service status is updated in real time, and customers receive notifications regarding their vehicle's progress.

d. Invoice Management

- Invoices are generated for each service, detailing the labor, parts used, and total costs.
- Customers can view and pay invoices through the mobile app, while mechanics and administrators manage invoices through the web interface.

e. Service Tracking

- This feature allows customers to track the progress of their vehicle's service in real time. Mechanics update the service status, and customers are notified of key milestones or delays.

4. API Endpoints

The Django REST API provides several endpoints for interacting with the system:

1. /api/users/

- **Description:** Manages user-related actions such as registration, authentication, profile management, and role-based access.
- **Methods:** GET, POST, PUT, DELETE

2. /api/cars/

- **Description:** Handles customer vehicle data, including details like make, model, year, and ownership history.
- **Methods:** GET, POST, PUT, DELETE

3. /api/service_types/

- **Description:** Provides service type information such as maintenance services (oil change, tire rotation) or repair services.
- **Methods:** GET, POST, PUT, DELETE

4. /api/tasks/

- **Description:** Manages tasks related to vehicle service jobs. Each task represents an individual work item (e.g., oil change, brake inspection).
- **Methods:** GET, POST, PUT, DELETE

5. /api/quotes/

- **Description:** Generates and retrieves quotes for vehicle services, providing customers with an estimate of costs.
- **Methods:** GET, POST, PUT, DELETE

6. /api/services/

- **Description:** Tracks the actual services being performed on vehicles, including their status (In Progress, Completed).
- **Methods:** GET, POST, PUT, DELETE

7. /api/parts/

- **Description:** Manages the parts inventory, allowing mechanics to log and track which parts are used for each service.
- **Methods:** GET, POST, PUT, DELETE

8. /api/invoices/

- **Description:** Handles invoice generation, payment status, and cost breakdown for completed services.
- **Methods:** GET, POST, PUT, DELETE

9. /api/appointments/

- **Description:** Manages customer service appointments, including booking, modification, and cancellation.
- **Methods:** GET, POST, PUT, DELETE

10. **/api/carhealth/**

- **Description:** Tracks car health data based on mileage, service history, and inspection results. Can be used to predict maintenance needs.
- **Methods:** GET, POST, PUT, DELETE

11. **/api/skillsharing/**

- **Description:** Allows mechanics to share skills and knowledge, creating a collaborative environment within the workshop.
- **Methods:** GET, POST, PUT, DELETE

12. **/api/taskreport/**

- **Description:** Provides reports on specific tasks completed for a vehicle, detailing the work done, and parts used.
- **Methods:** GET, POST

13. **/api/servicereport/**

- **Description:** Generates reports on services performed for a customer, giving a comprehensive history of all work completed on a vehicle.
- **Methods:** GET, POST

Each endpoint uses HTTP methods such as `GET`, `POST`, `PATCH`, and `DELETE` for data operations, ensuring RESTful principles are adhered to.

5. Database Structure

The database schema is designed to represent the various entities and their relationships, this is a brief overview of each model:

1. **User, Customer, Mechanic, Manager:**

- These models manage different types of users in the system. User is the base model, extended by Customer, Mechanic, and Manager models.
- They include functionality for password hashing and role management.

2. **PredefinedCar, Car:**

- PredefinedCar stores a list of car makes, models, and years, which is referenced by the Car model, associated with a specific customer.

- Car also tracks availability and registration.

3. **CarHealth:**

- Stores health data for a car, such as the odometer reading, battery level, tires condition, etc., updated regularly to monitor car maintenance.

4. **ServiceType:**

- Stores predefined types of services, each with a name, description, and labor cost.

5. **PredefinedTask, Task:**

- PredefinedTask refers to the predefined list of tasks for a particular service type.
- Task records individual tasks assigned to a mechanic for a specific service, tracking their progress and status.

6. **Service:**

- Represents a service performed on a car, associated with a specific service type and status, which progresses from requested to completed.

6. **TaskReport, ServiceReport:**

- TaskReport logs comments, images, and status of individual tasks.
- ServiceReport compiles multiple task reports into an overview of a service performed.

7. **PredefinedPart, Part:**

- PredefinedPart defines parts needed for services, linked to a service type.
- Part is linked to a specific car and records price details.

8. **Quote, QuotePart:**

- Quote provides a breakdown of parts and labor for a service, with a status to track customer acceptance.
- QuotePart links parts to the quote, defining the quantity and price for each part.

9. **Invoice, PartInvoice:**

- Invoice generates a payment record for services, tracking labor, taxes, discounts, and the total amount.
- PartInvoice links parts and their associated costs to the invoice.

10. **Appointment:**

- Schedules services for a specific date and time, with status tracking.

6. **Security Considerations**

- Data Encryption: All sensitive data, such as passwords and payment details, are encrypted.

- **Role-Based Access:** The system ensures that only authorized users can access or modify certain data, according to their role

1. customers cannot access mechanic functions
2. mechanics cannot access customer functions
3. only the manager can access the web app

7. Deployment & Maintenance

- **Containerization:** The system components (backend, frontend, and database) are containerized using Docker for ease of deployment and scalability.

8. Conclusion

The AutoWorx Barnyard Workshop Management System is built on a robust, scalable architecture, combining a mobile app, web app, and backend API to offer a complete solution for workshop operations. The modular nature of the system makes it easy to extend or modify functionality as needed. The integration of service tracking, appointment scheduling, and invoice management ensures a seamless experience for both customers and workshop staff.